Milestone 3: [20 points] Cache integration and performance analysis

In milestone 3, you need to further improve your simulator (on top of milestone 2) to simulate a realistic five-stage CPU pipeline together with a one-level cache system (based on lab 4). Here we only consider the data cache and do not model the instruction cache. That is, for load and store instructions, in the memory access stage of the pipeline, you will have to simulate their data cache accesses, including cache hits, misses, and evictions, with specified hit and miss latencies. Moreover, you will analyze the impact of cache configurations on program performance.

Framework Code Modifications

We provide you with framework code updates (project_ms3.zip) to help you keep one unified source code to support all milestones. In this milestone, we have given you five updated files including ***cache.c, cache.h, config.h, utils.h, riscv.c,*** which have minor changes from the previous ones. Please replace these five files with the newly updated ones from project_ms3.zip.

Please follow the instructions to get the milestone 3 files appropriately:

1. Replace ***cache.c, cache.h, config.h, utils.h, riscv.c*** under your project folder
2. Copy the **ms3** folder into the **code** folder.
3. Copy test_simulator_ms3.sh under your project folder.
4. Change your pipeline.h: after the line "extern uint64_t total_cycle_counter;", add "extern uint64_t mem_access_counter;".
5. Change your pipeline.c: after the line "uint64_t total_cycle_counter = 0;", add "uint64_t mem_access_counter = 0;".

Your Implementation

In lab 4, you have already developed a single-level data cache simulator. In this milestone, your major task is to integrate your lab 4 code into the final project. For better integration, we have updated baseline ***cache.c*** and ***cache.h*** files. ***cache.h*** includes all the cache configurations and cache hit/miss/eviction latencies, which are defined in macros.

First, for ***cache.c***, you need to add your lab 4 code to ***cache.c***, where it has been instructed as **/*YOUR CODE HERE*/**. Note in ***riscv.c***, we have the L1 cache set up before simulating the processor and deallocated the cache after the simulation. These steps are already taken care of, and you do not need to change the ***riscv.c***.

Second and most important, when simulating load and store instructions, you need to call the cache simulator during the memory access stage of the RISC-V processor pipeline (in your cycle-accurate simulator from milestone 2). The entry point to the cache simulator is ***processCacheOperation****(unsigned long* ***address****, Cache* ****cache****)*. Note that your cache simulator from lab 4 is for 64-bit processors (i.e., 64-bit memory addresses); in this project, you need to adapt it to the 32-bit RISC-V processor (i.e., 32-bit memory addresses). Here the cache simulator is supposed to get a 32-bit memory address from the instruction pipeline (memory access stage)

and return the access latency. Similar to lab 4, we only simulate the cache access latency and do not simulate the actual data in the cache.

For the access latency, let's consider 2 cycles for cache hit and 100 cycles for cache miss penalty (i.e., memory access latency; a cache eviction has the same cache miss penalty), which are defined as macros in ***cache.h***. **Note** the total cache miss latency is the sum of cache hit latency and miss penalty, which is 102 cycles. Please use those macros instead of hard-coding them in your code, so that it's flexible for you to change those latency configurations. To simulate the execution cycles, we simply need to add the returned cache access latency to the global cycle counter. Note the cache access delays don't cause any actual pipeline hazard, as the memory writes and reads both happen in the memory access stage; if you draw a pipeline stage diagram for two dependent memory access instructions (e.g., a pair of store and load instructions), you will see there is no pipeline hazard. So, we don't need any hazard detection/resolving to deal with cache accesses; simply add the returned cache access latency to the global cycle counter of your simulator.

- In case of cache hit, increase the global cycle counter by the cache hit cycles (i.e., 2 cycles in our configuration). **Note** since you already increase the global cycle counter by 1 inside the ***cycle_pipeline*** function in ***pipeline.c***, in the memory access stage, you should increase the global cycle counter by (cache hit latency – 1) cycles, i.e., 1 cycle.
- In case of cache miss, increase the global cycle counter by the cache miss cycles, which is the sum of cache hit latency and miss penalty (i.e., memory access latency); that is, 102 cycles in our configuration. **Note** since you already increase the global cycle counter by 1 inside the ***cycle_pipeline*** function in ***pipeline.c***, in the memory access stage, you should increase the global cycle counter by (cache miss latency – 1) cycles, i.e., 101 cycles.
- A cache eviction is also a cache miss, so also increase the global cycle counter by the cache miss cycles.

Moreover, in milestone 3, since we are considering the memory access latency, you need to add a latency of 100 cycles for every memory access if the cache is not enabled. A macro is already defined in ***config.h*** file for this purpose (***MEM_LATENCY***). Again, simply add this memory access latency to the global cycle counter of your simulator, when you simulate a processor without a cache. This is more realistic than milestone 2 where we didn't consider the memory access latency. This way, you can see the benefits of adding a cache to the processor.

- **Note** since you already increase the global cycle counter by 1 inside the ***cycle_pipeline*** function in ***pipeline.c***, in the memory access stage, you should increase the global cycle counter by (memory access latency – 1) cycles, i.e., 99 cycles.

Lastly, in addition to the previous prints in milestone 2, such as forwarding [FWD], flushing [CPL] and hazard handling [HZD], you will need to add some more prints for the cache access information like below.

[MEM]: Cache miss for address: 0x00000404

[MEM]: Cache latency at addr: 0x00000404: 102 cycles

The first line is printed from operateCache() function in ***cache.c***, which is guarded by the ***PRINT_CACHE_TRACES*** macro (defined in config.h). Three printing formats

CACHE_{EVICTION, HIT, MISS}_FORMAT are already defined in ***utils.h***, which is included in cache.h. The second line is printed from stage_mem() function in ***pipeline.c***, which is guarded by the ***PRINT_CACHE_TRACES*** macro (defined in config.h). The reference trace files are provided as well.

Milestone 3 Testing

To evaluate your cache integration, we have provided you with a set of input instruction traces in "./code/ms3/input/". Note that you **should not** change the cache configuration in the ***cache.h*** for test category 1 and 2; otherwise, you will get different traces.

Test category 1 (4 points):

Important Note #1: For the **test category 1** you need to enable all the macros in ***config.h*** file for milestone 3 as follows and disable the rest macros for milestone 1 and 2:

```
// required for MS3: (test_simulator_ms3.sh)
#define DEBUG_REG_TRACE
#define DEBUG_CYCLE
#define PRINT_STATS
#define MEM_LATENCY 100
#define CACHE_ENABLE              // enable cache simulation
#define PRINT_CACHE_STATS     // prints the cache stats at the end of program
#define PRINT_CACHE_TRACES  // prints the cache trace for each memory access
```

Commands to run:

1. L/S type instructions with cache (1 point)

```
./riscv -s -f -c -v  code/ms3/input/LS/LS.input > ./code/ms3/out/LS/LS.trace
diff ./code/ms3/ref/LS/LS.trace ./code/ms3/out/LS/LS.trace
```

2. Multiply set with cache (1 point)

```
./riscv -s -f -c -e -v ./code/ms3/input/multiply.input > ./code/ms3/out/multiply.trace
diff ./code/ms3/ref/multiply.trace ./code/ms3/out/multiply.trace
```

3. Random set with cache (1 point)

```
./riscv -s -f -c -e -v ./code/ms3/input/random.input > ./code/ms3/out/random.trace
diff ./code/ms3/ref/random.trace ./code/ms3/out/random.trace
```

4. testset_1 with cache (1 point)

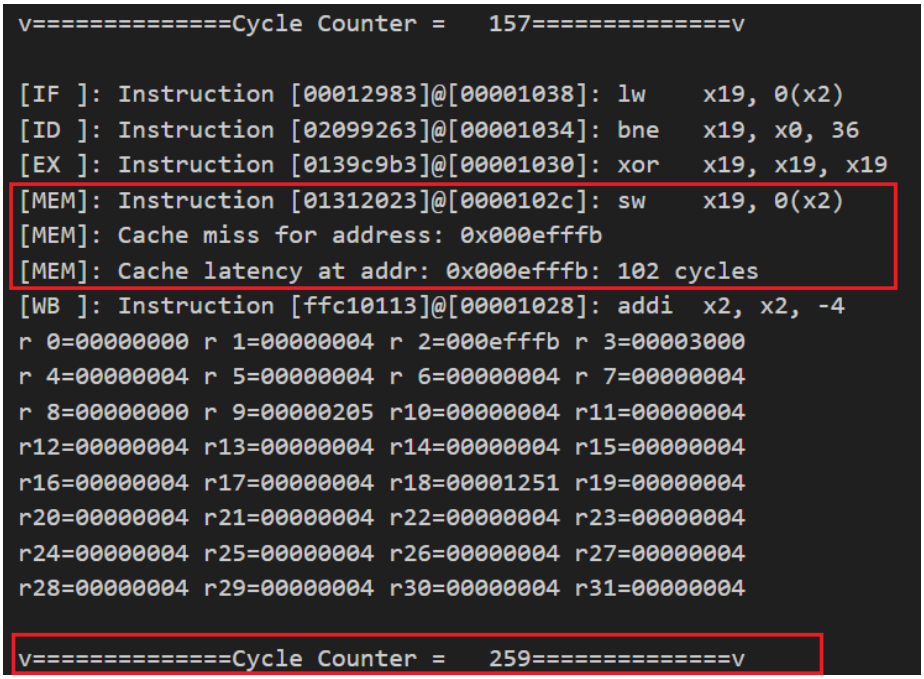
./riscv -s -f -c -e -v ./code/ms3/input/testset_1.input > ./code/ms3/out/testset_1.trace

diff ./code/ms3/ref/testset_1.trace ./code/ms3/out/testset_1.trace

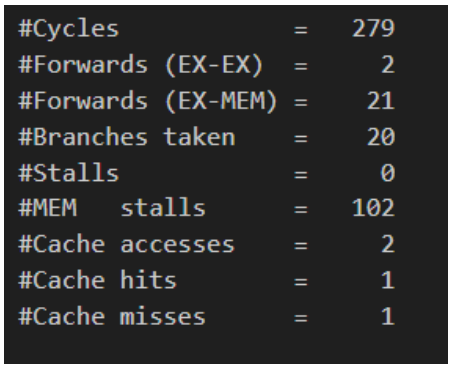
Note: You can run test category 1 with ***./test_simulator_ms3.sh cache_complete***

Example of a trace with cache in test category 1:

For a load or store instruction, here you can see the **MEM** stage has three printouts. The first printout is from milestone 2, and it is guarded by the DEBUG_CYCLE macro. The second one is printed from operateCache() function in ***cache.c*** guarded by the PRINT_CACHE_TRACES macro as explained above. The third printout is printed from stage_mem() function in ***pipeline.c*** guarded by the PRINT_CACHE_TRACES macro. In addition, you can see the global cycle counter is updated from 157 to 259 (instead of 158), due to 102 extra cycles cache miss latency.



At the end of each trace there should be stats as well, which show the total number of cache accesses, hits, misses, stalls caused by cache/memory access, in addition to the number of cycles, forwarding, branches and stalls (pipeline data hazard stalls).



Test category 2 (1 point):

Important Note #2: For the following tests you need to disable some macros in ***config.h*** file as follows due to tremendous size of output trace and disable the rest of the macros for milestone 1 and 2:

```
// required for MS3: (test_simulator_ms3.sh)
//#define DEBUG_REG_TRACE
//#define DEBUG_CYCLE
#define PRINT_STATS
#define MEM_LATENCY 100
#define CACHE_ENABLE            // enable cache simulation
#define PRINT_CACHE_STATS       // prints the cache stats at the end of program
//#define PRINT_CACHE_TRACES // prints the cache trace for each memory access
```

5. Vector cross-product set with cache (1 point)

./riscv -s -f -c -e -v ./code/ms3/input/vec_xprod.input > ./code/ms3/out/vec_xprod.trace

diff ./code/ms3/ref/vec_xprod.trace ./code/ms3/out/vec_xprod.trace

Note: You can run test category 2 with ***./test_simulator_ms3.sh cache_summary***

Example of a trace with cache in test category 2:

Test category 2 disables the trace for each instruction, since it becomes a tremendously large trace file. The trace just has the final stats. Please see the reference traces for more insights.

```
========
[MAIN]: Flushing pipeline
========
#Cycles            = 3769724
#Forwards (EX-EX)  = 767254
#Forwards (EX-MEM) = 485634
#Branches taken    = 78112
#Stalls            = 208128
#MEM   stalls      = 1493688
#Cache accesses    = 249088
#Cache hits        = 236642
#Cache misses      = 12446
```

Performance analysis (1 point)

Next, you are required to analyze the impact of cache on performance. The vec_xprod.input file implements an algorithm which is tiled to improve performance. To help you understand what the program does and its data access pattern, ./code/ms3/ref/vec_xprod.S file has been provided, which

includes a more readable disassembly of the program with comments. **You need to report the speedup gain with cache (against the one without cache) in your report.**

Commands to run with cache simulator:

./riscv -s -f **-c** -e -v ./code/ms3/input/vec_xprod.input > ./code/ms3/out/vec_xprod.trace

diff ./code/ms3/ref/vec_xprod.input ./code/ms3/out/vec_xprod.trace

Note: You can run these commands with ***./test_simulator_ms3.sh cache_summary***

Important Note #3: For the following test you need to **disable one more macro** in ***config.h*** file as follows and disable the rest of the macros for milestone 1 and 2:

```
// required for MS3: (test_simulator_ms3.sh)
//#define DEBUG_REG_TRACE
//#define DEBUG_CYCLE
#define PRINT_STATS
#define MEM_LATENCY 100
//#define CACHE_ENABLE            // enable cache simulation
#define PRINT_CACHE_STATS       // prints the cache stats at the end of program
//#define PRINT_CACHE_TRACES  // prints the cache trace for each memory access
```

Command to run without cache simulator:

./riscv -s -f -e -v ./code/ms3/input/vec_xprod.input > ./code/ms3/out/vec_xprod.nocache.trace

diff ./code/ms3/ref/vec_xprod.nocache.trace ./code/ms3/out/vec_xprod.nocache.trace

Note: You can run these commands with ***./test_simulator_ms3.sh no_cache***

Cache configuration exploration and performance analysis (4 points)

Your objective in this part is to explore the cache configurations to maximize the cache hit rate of the vec_xprod.input testcase with cache size 1K, 2K, 4K and 8K. There are no limitations on cache configs except the cache size.

Important Note #4: Each time you change the cache configuration, you need to make clean and make again with the following commands:

```
make clean
make
```

For each cache size, you need to **report** its best cache configuration, and the speedup versus the baseline without a cache. Also, provide 4 text files **dse_{1k, 2k, 4k, 8k}.txt** each for each cache size with the following format:

- In line 1, type **-s YOUR_SET_BITS**, e.g., **-s 3;**
- In line 2, type **-E YOUR_ASSOCIATIVITY,** e.g., **-E 4;**
- In line 3, type **-L** for LRU or **-F** for LFU**;**
- In line 4, type **-b YOUR_BLOCK_BITS**, e.g., **-b 4;**

Do not include anything else in these files; violating this will get 0 point for this part.

An example .txt file is shown below:

```
-s 3;
-E 4;
-F;
-b 4;
```

Milestone 3 Marking (20 points)

- Milestone 3 contains 20 points of the project.
- 5 points are given for the successful completion of **test category 1 and 2**. There are 5 tests in these two test categories: each test is worth 1 point. For each test, if the entire test passes (i.e., match with the reference output), you will earn 1 point; otherwise, you will get 0 points for the test. No partial points will be given.
- 1 point is given for the successful completion of the **performance analysis**. If the entire test passes (i.e., match with the reference output), you will earn 1 point; otherwise, you will get 0 points for the test. No partial points will be given. Note you are required to report the speedup as well.
- 4 points are given for **cache configuration exploration**. If your **dse_{1k, 2k, 4k, 8k}.txt** matches the final generated files (not given in this milestone), you will earn 1 point per file; otherwise, you will get 0 points for the file. No partial points will be given.
- 5 points are given for the **demo** of this part. Earning these points depends on how confident you will explain and answer questions about your work related to this part during the demo.
- 5 points are given for your **report**. You must include a section clearly describing how you have implemented milestone 3 with reference to your project code including the following details. If this section is missing, or the description is vague or incomplete, no points will be awarded.
    - Detailed manual calculations of number of cycles per instruction including memory and cache latency for the tests 1 – 4 in test category 1.
    - Demonstrate whether cache has improved the performance or not? And why?
    - Report the speedup gain with cache for vec_xprod.input. Report cache configuration exploration with an explanation of 4 text files **dse_{1k, 2k, 4k, 8k}.txt,** each for one cache size with the aforementioned format. Explain why these cache configurations show the highest hit rate per cache size on vec_xprod.input based on the information you could get from vec_xprod.S.